İSTANBUL BEYKENT ÜNİVERSİTESİ

Mühendislik - Mimarlık Fakültesi | Yazılım Mühendisliği Bölümü

WEB UYGULAMA GELİŞTİRME

Final Projesi Raporu

ASP.NET Core MVC / RESTful API Uygulaması

Öğrenci Adı ve Soyadı	Mert Uğur
Öğrenci Numarası	2203013016
Ders Adı	Web Uygulama Geliştirme
Sınıf / Şube	Şube 1
Proje Konusu / Adı	DietCok: Kalori Takip Sistemi
Öğretim Elemanı	Dr. Öğr. Üyesi Umut KAYA
Teslim Tarihi	11 Haziran 2026, Perşembe

1. Giriş

1.1. Gerçek Dünya Problemi Tanımı

Günümüzde birçok kişi sağlıklı beslenmek, kilo vermek, kilo almak veya mevcut kilosunu korumak amacıyla günlük beslenme düzenini takip etmeye çalışmaktadır. Ancak bu süreçte en çok karşılaşılan problemlerden biri, gün içinde tüketilen besinlerin düzenli ve doğru şekilde kaydedilememesidir. Kişiler çoğu zaman ne yediklerini tam olarak hatırlayamamakta, aldıkları kalori miktarını ve protein, karbonhidrat, yağ gibi makro besin değerlerini net olarak görememektedir.

Bu durum özellikle belirli bir hedefe göre beslenmeye çalışan kullanıcılar için sorun oluşturmaktadır. Çünkü kullanıcı yalnızca ne yediğini bilmekle kalmamalı, aynı zamanda aldığı değerlerin kendi günlük hedeflerinin altında mı yoksa üstünde mi olduğunu da görebilmelidir. Aksi halde beslenme süreci tahmine dayalı ilerlemekte ve kişi hedeflerine ne kadar yaklaştığını anlamakta zorlanmaktadır. Mevcut beslenme takip yöntemlerinde kullanıcılar genellikle bilgileri dağınık şekilde not almakta ya da sadece günlük tüketimlerine odaklanmaktadır. Bu nedenle haftalık veya aylık olarak genel beslenme durumunu değerlendirmek zorlaşmaktadır. Ayrıca benzer isimlere sahip besinler arasından doğru ürünü seçmek her zaman kolay olmamakta, bu da hatalı veri girişlerine neden olabilmektedir.

Bu proje kapsamında ele alınan temel problem, bireylerin günlük beslenme verilerini düzenli, anlaşılır ve hedefleriyle karşılaştırılabilir şekilde takip edememesidir. Bu problemin çözülmesiyle kullanıcıların beslenme alışkanlıklarını daha bilinçli şekilde değerlendirmesi ve hedeflerine yönelik daha kontrollü kararlar alması amaçlanmaktadır. Bu yapı önceki DietCok gereksinim raporundaki problem tanımıyla da uyumludur; orada da kullanıcıların tüketimlerini, kalori ve makro değerlerini düzenli şekilde takip etmekte zorlandığı belirtilmişti.

1.2. Projenin Hedefleri

Hedef 1: Kullanıcıların sisteme kayıt olarak kendi hesapları üzerinden beslenme takibi yapabilmesini sağlamak.

Hedef 2: Kullanıcıların günlük kalori, protein, karbonhidrat ve yağ hedeflerini sisteme girebilmesini sağlamak.

Hedef 3: Kullanıcıların gün içerisinde tükettikleri besinleri kahvaltı, öğle, akşam ve ara öğün gibi öğünlere göre kaydedebilmesini sağlamak.

Hedef 4: Kullanıcının eklediği besinlere göre günlük toplam kalori ve makro besin değerlerini otomatik olarak hesaplamak.

Hedef 5: Kullanıcının aldığı kalori ve makro değerleri, önceden belirlediği hedeflerle karşılaştırarak eksik veya fazla kalan değerleri göstermek.

Hedef 6: Kullanıcıların beslenme durumunu yalnızca günlük değil, haftalık ve aylık olarak da takip edebilmesini sağlamak.

Hedef 7: Takvim görünümü sayesinde kullanıcıların hangi günlerde hedeflerine uygun beslendiğini, hangi günlerde hedefin altında veya üstünde kaldığını daha anlaşılır şekilde göstermek.

Hedef 8: Besinlerin görsellerle desteklenmesini sağlayarak kullanıcıların doğru besini daha kolay seçmesine yardımcı olmak.

Hedef 9: Sistemde bulunmayan besinlerin kullanıcı tarafından eklenebilmesini sağlayarak beslenme kaydının eksik kalmasını önlemek.

Hedef 10: Kullanıcının beslenme verilerini daha düzenli ve anlaşılır hale getirerek, kendi beslenme alışkanlıklarını daha bilinçli şekilde değerlendirmesine yardımcı olmak.

1.3. Projenin Amacı

Bu projenin amacı, bireylerin günlük beslenme düzenlerini daha düzenli, anlaşılır ve hedef odaklı bir şekilde takip edebilmelerini sağlayan bir sistem tasarlamaktır. Günümüzde birçok kullanıcı kilo vermek, kilo almak, kilosunu korumak veya daha sağlıklı beslenmek istemektedir. Ancak gün içinde tüketilen besinlerin kalori, protein, karbonhidrat ve yağ değerlerini düzenli olarak takip etmek her zaman kolay olmamaktadır. Bu nedenle proje, kullanıcıların tükettikleri besinleri öğünlere göre kaydedebileceği, günlük hedefleriyle karşılaştırabileceği ve genel beslenme

durumunu daha net görebileceği bir yapı sunmayı amaçlamaktadır. Bu sistem sayesinde kullanıcılar kendi günlük kalori ve makro besin hedeflerini belirleyebilecek, gün içinde ekledikleri besinlerin toplam değerlerini görebilecek ve hedeflerinin altında mı yoksa üstünde mi kaldığını anlayabilecektir. Ayrıca takvim yapısı ile günlük, haftalık ve aylık beslenme durumunun izlenmesi hedeflenmektedir. Böylece kullanıcı yalnızca o gün ne yediğini değil, genel beslenme alışkanlığının nasıl ilerlediğini de değerlendirebilecektir.

Bu ders kapsamında ise projenin yalnızca fikir olarak değil, kalite açısından da değerlendirilebilir bir sistem haline getirilmesi amaçlanmaktadır. Proje sayesinde yazılımda kalite kavramını, kullanıcı ihtiyaçlarına uygunluk, hata risklerini belirleme, test edilebilirlik, güvenilirlik ve kullanılabilirlik gibi konular üzerinden öğrenmek hedeflenmektedir. Bu çalışma, bir yazılım sisteminin sadece çalışmasının yeterli olmadığını; aynı zamanda doğru, güvenilir, anlaşılır ve kullanıcı beklentilerine uygun şekilde tasarlanması gerektiğini göstermektedir. Proje bana, gerçek bir kullanıcı problemini yazılım kalite güvencesi bakış açısıyla analiz etme ve daha düzenli bir sistem tasarlama konusunda katkı sağlayacaktır.

2. Literatür

2.1. ASP.NET Core Framework

ASP.NET Core, Microsoft tarafından geliştirilen, modern ve platformlar arası web uygulamaları geliştirmeye yönelik açık kaynaklı bir çatıdır.

Benim projemde ASP.NET Core, geliştirdiğim Kalori Takip uygulamasının ana iskeleti ve beyni görevini üstlenmektedir. Kullanıcının web tarayıcısı üzerinden yaptığı kayıt olma, sisteme giriş yapma, günlük yediği yemekleri panele ekleme veya silme gibi tüm işlemler bu altyapı sayesinde kontrol edilmektedir. Projede ASP.NET Core'un MVC (Model-View-Controller) yapısını kullanarak arayüz ile arka plandaki veritabanı işlemlerinin birbiriyle uyum içinde çalışmasını sağladım. Uygulamayı bilgisayarımda yerel olarak başlattığımda, ASP.NET Core'un kendi içinde gelen yerel web sunucusu Kestrel devreye girmekte ve projeyi tarayıcı üzerinden test edebilmem için yayına almaktadır.

2.2. MVC Mimarisi

MVC (Model-View-Controller), uygulamayı üç temel katmana ayıran bir yazılım mimarisidir.

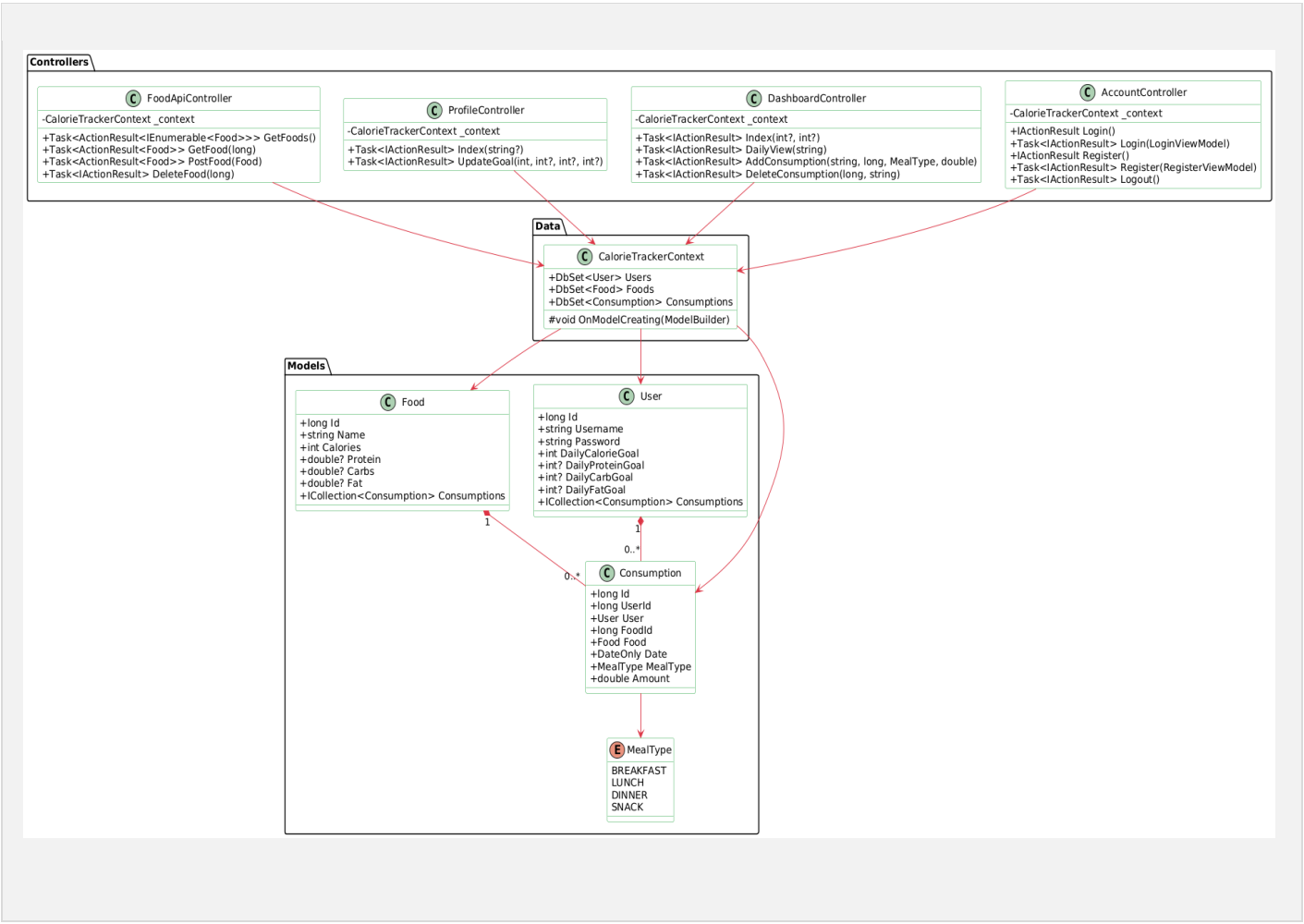
Katman	Görev	Projemde Kullandığım Sınıf
Model (Entity)	Verileri ve iş kurallarını temsil eder	User.cs, Food.cs, Consumption.cs
Controller	Kullanıcı isteklerini karşılar	DashboardController.cs, AccountController.cs, ProfileController.cs, FoodApiController.cs
View (Razor/HTML)	Kullanıcıya arayüz sunar	Views/Dashboard/Index.cshtml, Views/Dashboard/DailyView.cshtml, Views/Account/Login.cshtml

Service	İş mantığını yürütür	Projede ek bir servis sınıfı yerine iş mantığı doğrudan kontrolcülerin (örneğin DashboardController.cs içindeki aylık takvim matrisi hesaplamaları) içinde yürütülmüştür.
Repository	Veritabanı erişimini sağlar	CalorieTrackerContext.cs (Entity Framework Core DbContext yapısı, projede veritabanı sorgulama ve kaydetme deposu olarak görev yapar).

2.3. Dependency Injection

Dependency Injection (Bağımlılık Enjeksiyonu) nesnelerin bağımlılıklarını kendileri oluşturmak yerine dışarıdan almasını sağlayan bir tasarım desenidir. Benim projemde veritabanı bağlantımızı yöneten CalorieTrackerContext nesnesini kontrolcülerimize bağlarken Dependency Injection kullandım. Bu veritabanı nesnesini kontrolcü sınıfların içinde tek tek new anahtar kelimesiyle oluşturmak yerine DashboardController ve AccountController gibi sınıfların kurucu (constructor) metoduna parametre olarak enjekte ettim. Örneğin, DashboardController sınıfının kurucu metodunda veritabanı nesnemizi bu şekilde alarak kullanıcıların günlük kalori verilerini doğrudan sorgulayabildim. Bu sayede kodlarımızın veritabanı bağlantı detaylarına olan bağımlılığını azaltmış oldum.

3. UML Sınıf Diyagramı



Şekil 3: UML Sınıf Diyagramı — Calorie Tracker (DietCok)

Diyagramda User (Kullanıcı) sınıfı ile Consumption (Tüketim) sınıfı arasında bire-çok (one-to-many) ilişki bulunmaktadır; yani bir kullanıcı birden fazla yemek tüketimi kaydedebilir. Benzer şekilde Food (Besin) sınıfı ile Consumption sınıfı arasında da bire-çok ilişki vardır. Consumption sınıfı, her bir tüketim kaydı için hangi öğünün seçildiğini MealType enumu (Kahvaltı, Öğle, Akşam, Ara Öğün) üzerinden tutmaktadır. Tüm veritabanı işlemlerini yöneten CalorieTrackerContext (DbContext) sınıfı, modellerimize (User, Food, Consumption) erişim sağlamaktadır. Uygulamadaki AccountController, DashboardController, ProfileController ve FoodApiController sınıfları ise veritabanı işlemlerini gerçekleştirebilmek amacıyla bağımlılık enjeksiyonu üzerinden CalorieTrackerContext sınıfına erişmektedir.

4. Projede Kullanılan Teknolojiler

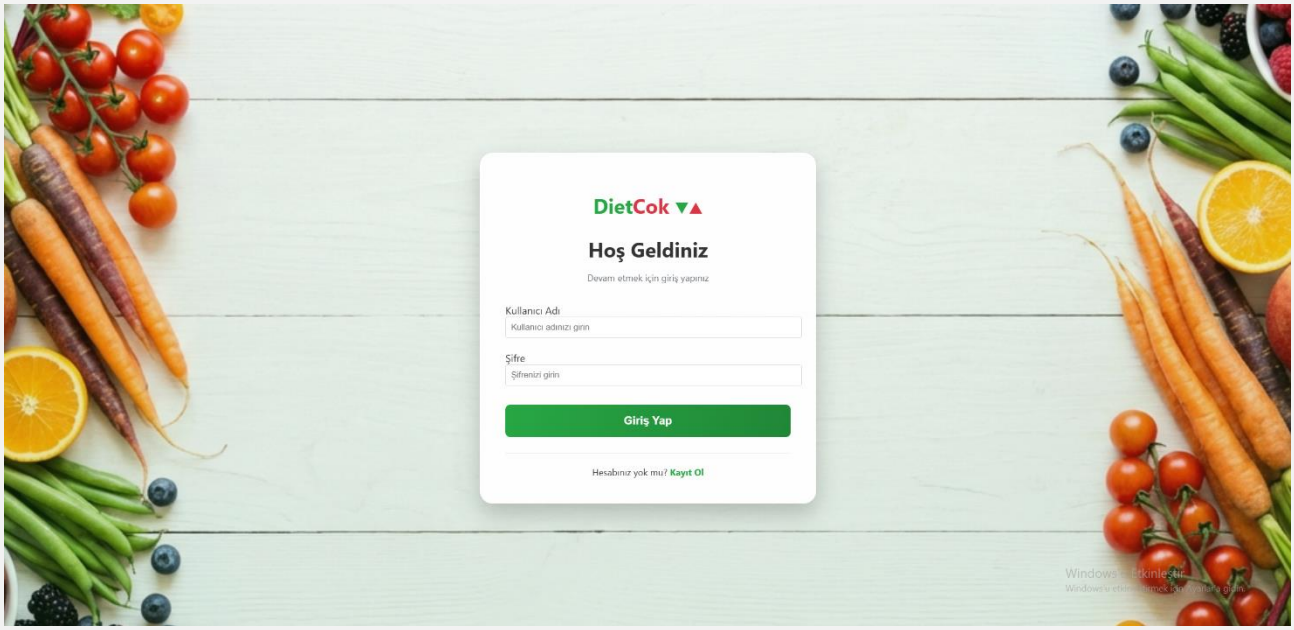
Teknoloji / Araç	Kullandığım Versiyon	Kullanım Amacı / Gerekçe
ASP.NET Core	.NET 8.0	Yüksek performanslı, güvenli ve platformlar arası modern bir geliştirme ortamı sunduğu için tercih edilmiştir. Projede, kullanıcı oturumlarının çerez tabanlı yönetilmesi, gelen web isteklerinin ilgili kontrolcülere yönlendirilmesi ve kalori/makro hesaplama gibi temel backend iş mantığının yürütülmesi amacıyla kullanılmıştır.
IDE (Visual Studio / VS Code / Rider)	Visual Studio 2022	C# ve .NET projelerinde güçlü kod tamamlama, kolay hata ayıklama (debugging) ve görsel yönetim araçları sunduğu için tercih edilmiştir. Proje kodlarının yazılması, veritabanı şemalarının oluşturulması ve uygulamanın derlenip yerel sunucuda çalıştırılması sürecinde kullanılmıştır.
Veritabanı (SQL Server / SQLite / PostgreSQL)	SQLite 3	Ekstra bir veritabanı sunucusu kurulumu gerektirmemesi ve veritabanını tek bir dosya halinde proje klasöründe tutarak taşınabilirliği kolaylaştırması nedeniyle seçilmiştir. Kullanıcı hesap bilgilerinin, sistemdeki hazır besinlerin ve kullanıcıların günlük öğün tüketim kayıtlarının kalıcı olarak saklanması amacıyla kullanılmıştır.
Entity Framework Core	8.0.0	Karmaşık SQL sorguları yazmak yerine nesne yönelimli C# sınıfları üzerinden veritabanı işlemlerini daha güvenli ve hızlı yönetebilmek (ORM) amacıyla tercih edilmiştir. Projedeki veritabanı tabloları ile modellerimiz arasında bağlantı kurarak veri ekleme, listeleme ve silme gibi işlemleri gerçekleştirmek için kullanılmıştır.
Razor Views	Razor Engine	Sunucu tarafında hazırlanan dinamik verileri HTML şablonlarıyla kolayca birleştirip kullanıcıya yansıtabildiği için seçilmiştir. Kullanıcının giriş ve kayıt sayfaları, aylık özet takvimi, günlük öğün günlüğü ve profil ayarları

		ekranlarının dinamik olarak oluşturulmasında kullanılmıştır.
NuGet / dotnet CLI	.NET CLI 8.0	Proje bağımlılıklarını kolayca yönetebilmek ve projeyi komut satırından hızlıca build edebilmek amacıyla tercih edilmiştir. Projede Entity Framework Core, SQLite ve Swagger gibi paketlerin kurulması ve uygulamanın terminal üzerinden başlatılması süreçlerinde kullanılmıştır.
Postman	Postman v10	Yazılan RESTful API metotlarını tarayıcıdan bağımsız olarak, farklı HTTP istekleri ve parametreler ile detaylıca test edebilmek amacıyla seçilmiştir. Besin yönetimini sağlayan API uç noktalarımızın (endpoints) doğru yanıtlar döndürdüğünü doğrulamak için kullanılmıştır.
Swagger / OpenAPI (varsa)	Swashbuckle 6.5.0	Geliştirilen API servislerini otomatik olarak standart bir formatta belgelemek ve interaktif bir arayüzle test etmeyi kolaylaştırmak için seçilmiştir. Tarayıcı üzerinden /swagger rotasında API metotlarını görselleştirmek ve yazılan servisleri doğrudan test etmek amacıyla kullanılmıştır.

5. Uygulamanın Ekran Görüntüleri ve Açıklamaları

5.1. Projede Gerçekleştirilenler

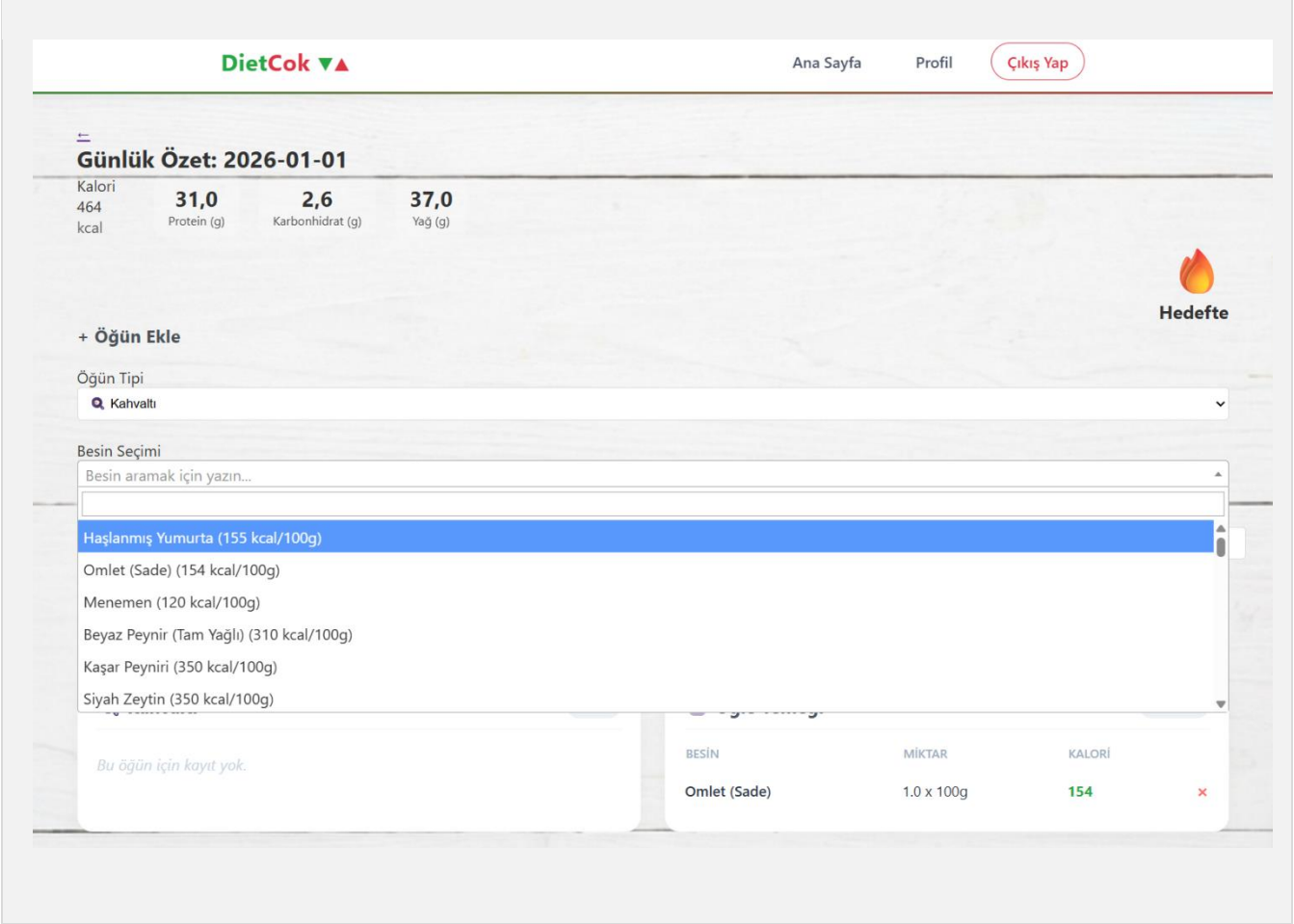
5.1.1. Ana Sayfa / Giriş Ekranı



Şekil 2: Ana Sayfa / Giriş Ekranı

Uygulama ilk açıldığında kullanıcıyı karşılayan giriş ekranıdır. Sayfanın arka planında projenin diyet ve kalori takibi konseptini destekleyen şık bir sebze ve meyve görseli yer almaktadır. Ekranın merkezinde konumlandırılmış form alanında, kayıtlı kullanıcıların sisteme giriş yapabilmesi için kullanıcı adı ve şifre giriş alanları bulunmaktadır. Kullanıcı geçerli bilgilerini girip yeşil renkli 'Giriş Yap' butonuna bastığında kimlik doğrulaması yapılarak doğrudan ana kontrol paneline (Dashboard) yönlendirilir. Eğer kullanıcının sisteme kayıtlı bir hesabı bulunmuyorsa, alt kısımda yer alan 'Kayıt Ol' bağlantısına tıklayarak yeni hesap oluşturma ekranına kolayca geçiş yapabilir.

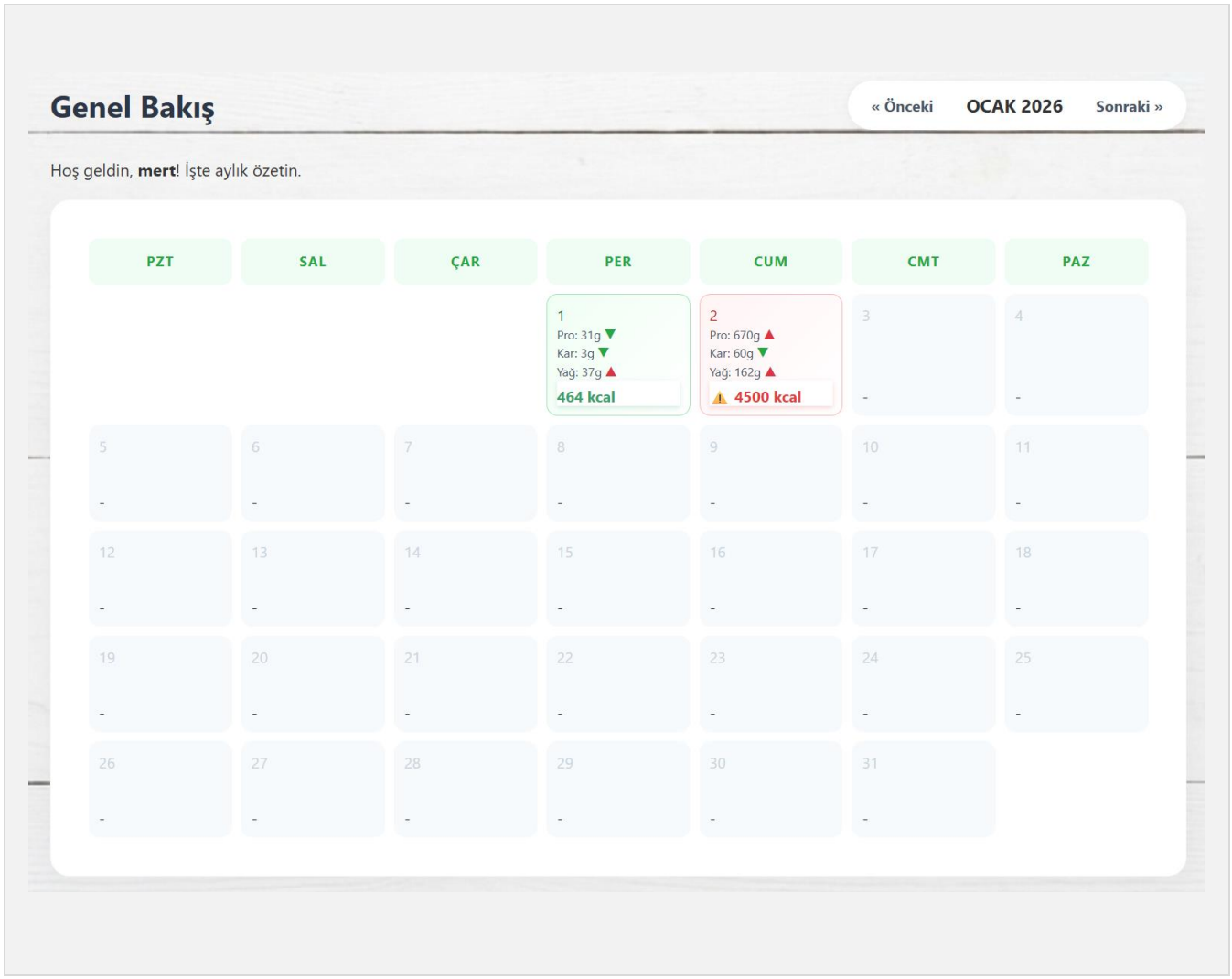
5.1.2. Kayıt Ekleme (Create)



Şekil 3: Yeni Kayıt Ekleme Formu

Sisteme giriş yapmış olan kullanıcının, günlük yemek tüketimlerini veritabanına kaydettiği "Öğün Ekle" arayüzüdür (Create işlemi). Görselde, kullanıcının Kahvaltı öğünü için besin seçimi yaptığı ve arama kutusunun açık olduğu an gösterilmektedir. Arama kutusuna harf girilmeye başlandığında, Select2 kütüphanesi sayesinde veritabanında kayıtlı olan besinler (Haşlanmış Yumurta, Omlet, Menemen vb.) otomatik olarak filtrelenerek listelenir. Kullanıcı porsiyon miktarını (100 gramın katları olacak şekilde, örneğin 1.0) girip formu gönderdiğinde, arka planda yeni bir tüketim kaydı oluşturulur. Eklenen veriler alt kısımdaki ilgili öğün kartlarında anlık olarak listelenirken, üst paneldeki günlük toplam kalori ve makro (protein, karbonhidrat, yağ) değerleri de otomatik olarak güncellenir.

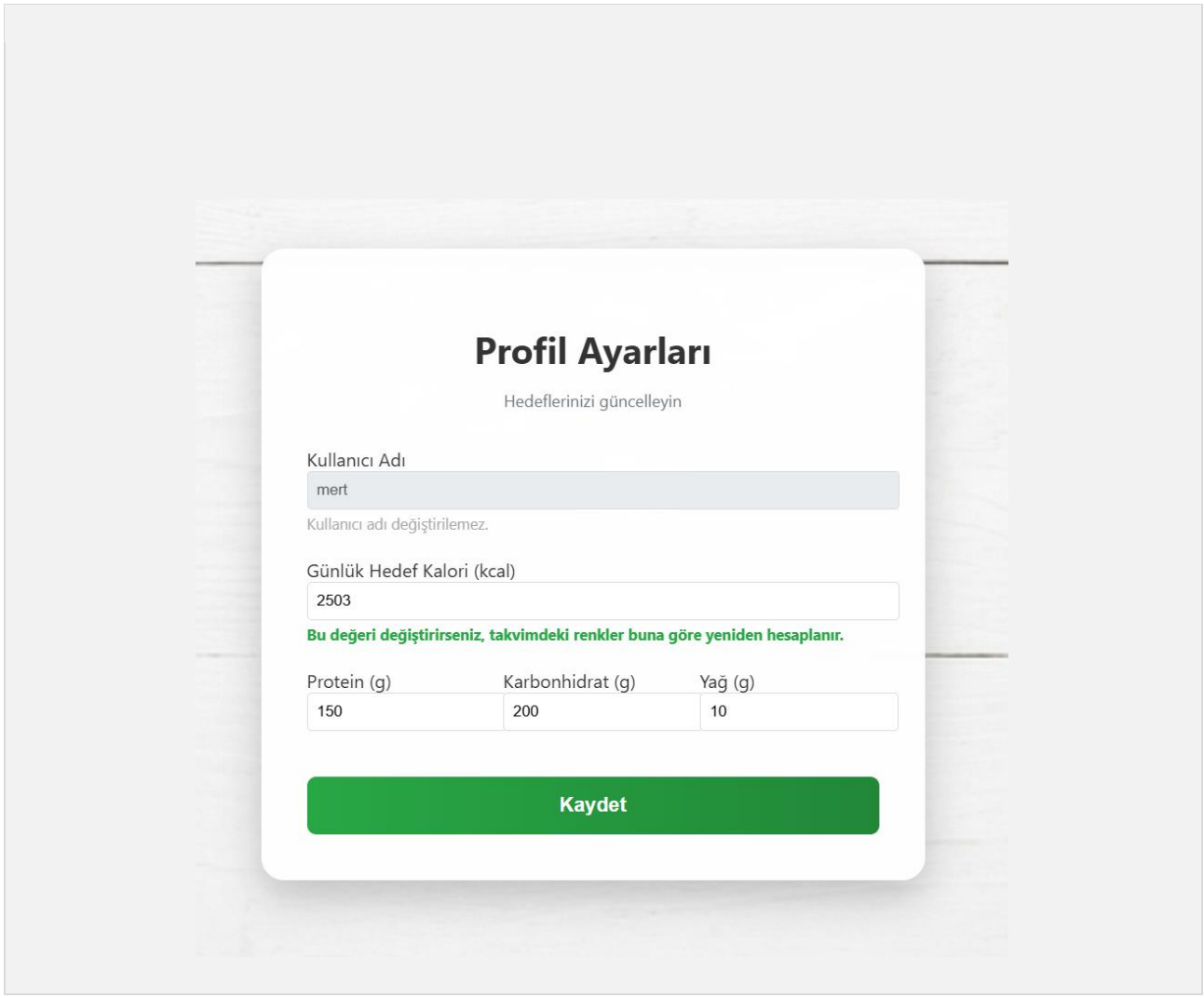
5.1.3. Kayıtları Listeleme (Read)



Şekil 4: Kayıt Listeleme Ekranı (Dashboard)

Veritabanına kaydedilen tüm tüketim verilerinin aylık bazda toplu olarak listelendiği "Genel Bakış" (Dashboard) ekranıdır (Read işlemi). Veritabanından çekilen besin tüketim kayıtları, ilgili günlere ait toplam kalori ve makro (protein, karbonhidrat, yağ) değerleri hesaplanarak takvim üzerinde dinamik olarak listelenir. Sistem, kullanıcının profilinde tanımladığı günlük kalori ve makro hedeflerini referans alarak karşılaştırma yapar. Günlük kalori hedefinin altında kalınan günler yeşil renkle (Örn: 1 Ocak), hedefin aşıldığı günler ise kırmızı renk ve uyarı sembolüyle (Örn: 2 Ocak) gösterilir. Tüketim kaydı bulunmayan günler ise sade ve gri renkte listelenmektedir.

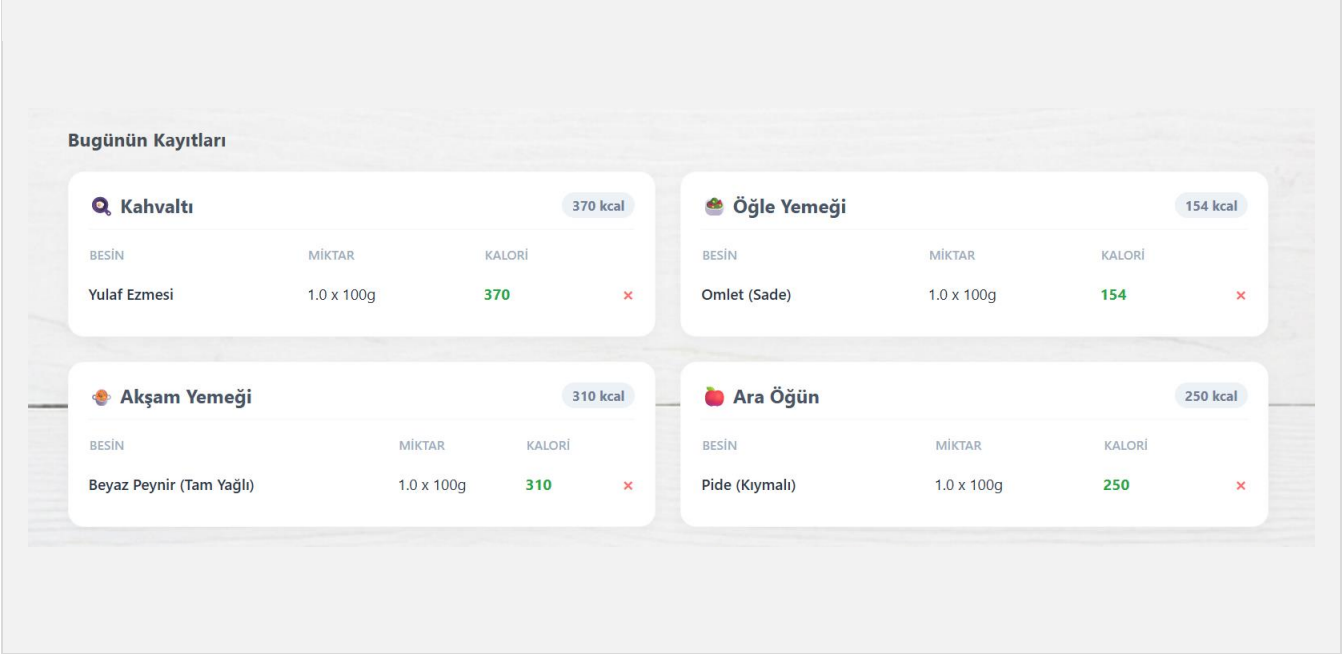
5.1.4. Kayıt Güncelleme (Update)



Şekil 5: Kayıt Güncelleme Ekranı

Kullanıcının günlük kalori ve makro (protein, karbonhidrat, yağ) hedeflerini güncellediği profil ayarları ekranıdır (Update işlemi). Güvenlik sebebiyle kullanıcı adı değiştirilemeyecek şekilde kilitlenmiş (disabled), kalori ve makro hedef alanları ise düzenlemeye açık bırakılmıştır. Kullanıcı yeni hedeflerini (örneğin günlük 2503 kcal) yazıp yeşil renkli "Kaydet" butonuna bastığında, veritabanındaki kullanıcı kaydı güncellenir ve yeni hedefler sisteme anında yansıtılır.

5.1.5. Kayıt Silme (Delete)



Şekil 6: Kayıt Silme İşlemi

Kullanıcının gün içinde eklediği yiyecek kayıtlarını listesinden kaldırabildiği ekran görünümüdür (Delete işlemi). Görselde yer alan dört farklı öğün kartındaki her bir besin kaydının sağ tarafında kırmızı renkli bir çarpı (×) butonu yer almaktadır. Kullanıcı silmek istediği yiyeceğin yanındaki bu butona tıkladığında, ilgili kaydın benzersiz ID bilgisi arka plana gönderilerek veritabanından kalıcı olarak silinir. Kullanım kolaylığı sağlamak amacıyla ekstra bir onay penceresi gösterilmeden işlem doğrudan tamamlanır; silme işleminin ardından sayfa yenilenerek o günün toplam kalori ve makro değerleri güncel haliyle ekranda yeniden hesaplanır.

5.2. Projede Alınan Hatalar

#	Hata	Nedeni	Çözüm
1	Microsoft.Data.Sqlite.SqliteException: SQLite Error 14: 'unable to open database file'	Proje ilk kez çalıştırılırken, SQLite veritabanı dosyasının (calorietracker.db) oluşturulacağı dizine uygulamanın yazma izninin bulunmaması.	appsettings.json dosyasındaki Connection String tanımı göreli (relative) yol olarak "Data Source=calorietracker.db" şeklinde güncellenerek veritabanının doğrudan proje ana klasörü altında otomatik oluşturulması sağlanmıştır.

2	System.Text.Json.JsonException: A possible object cycle was detected.	RESTful API (FoodApiController) üzerinden besinler çekilirken, Food sınıfı ile Consumption sınıfı arasındaki çift yönlü ilişkiden dolayı JSON serileştiricinin sonsuz döngüye girmesi.	API kontrolcüsü içerisindeki GET metotlarında veritabanı nesnelerini doğrudan döndürmek yerine, LINQ Select metodu kullanılarak sadece gerekli alanları içeren yeni anonim nesneler (new { f.Id, f.Name }) oluşturulup döndürülmüş ve döngü kırılmıştır.
3	Uncaught ReferenceError: $ is not defined (Tarayıcı Konsol Hatası)	Sayfa şablonunda (_Layout.cshtml ve DailyView.cshtml) script yükleme sıralaması düzeltilerek jQuery kütüphanesi en üste taşınmış ve Select2 kodları jQuery yüklendikten sonra çalışacak şekilde konumlandırılmıştır.	Sayfa şablonunda (_Layout.cshtml ve DailyView.cshtml) script yükleme sıralaması düzeltilerek jQuery kütüphanesi en üste taşınmış ve Select2 kodları jQuery yüklendikten sonra çalışacak şekilde konumlandırılmıştır.
4	Microsoft.EntityFrameworkCore.DbUpdateException: Foreign key constraint failed	Veritabanında kayıtlı bir besin (Food) silinmek istendiğinde, o besine ait geçmiş tüketim kayıtlarının (Consumption) veritabanında askıda kalması ve yabancı anahtar (FK) kısıtlamasının ihlal edilmesi.	CalorieTrackerContext.cs dosyasındaki Fluent API yapılandırmasında OnDelete(DeleteBehavior.Cascade) kuralı eklenmiştir. Böylece bir besin silindiğinde, ona bağlı geçmiş tüketim kayıtlarının da otomatik olarak silinmesi sağlanmıştır.
5	System.InvalidOperationException: The DateOnly type is not supported by SQLite...	SQLite veritabanının, C# tarafında kullanılan DateOnly (tarih) veri tipini veritabanı seviyesinde nasıl saklayacağını varsayılan olarak çözümleyememesi.	Projedeki Entity Framework Core NuGet paketleri güncellenerek EF Core 8.0'ın yerleşik (native) DateOnly dönüştürücü desteği etkinleştirilmiş ve tarih verileri SQLite üzerinde sorunsuz şekilde metin tabanlı eşleştirilmiştir.

5.3. Projenin Değerlendirilmesi

Yapılanlar

1. Kullanıcı Kayıt ve Güvenli Giriş Sistemi: BCrypt.Net-Next kütüphanesi kullanılarak şifrelerin güvenli bir şekilde hash'lenmesi ve Cookie Authentication altyapısı ile kullanıcı giriş, kayıt ve güvenli çıkış işlemlerinin tamamlanması.
2. Dinamik Takvimli Ana Kontrol Paneli (Dashboard): Kullanıcının aylık kalori ve makro durumunu gün bazında özetleyen, kalori sınırının aşılıp aşılmadığını renk kodlarıyla (yeşil/kırmızı) gösteren aylık takvim arayüzünün oluşturulması.
3. Günlük Tüketim Günlüğü ve Öğün Yönetimi: Kahvaltı, Öğle Yemeği, Akşam Yemeği ve Ara Öğün olmak üzere 4 farklı kategoride besin ekleme, veritabanından dinamik arama (Select2) ve kayıt silme (CRUD) işlevlerinin başarıyla kodlanması.
4. Profil ve Hedef Yönetimi: Kullanıcının günlük kalori, protein, karbonhidrat ve yağ limitlerini dilediği zaman güncelleyebileceği ve tüm sistemde referans alınacak dinamik ayarlar ekranının tamamlanması.
5. RESTful API ve Swagger Entegrasyonu: Besin kütüphanesine dışarıdan erişim sağlayan API uç noktalarının (FoodApiController) yazılması ve projenin interaktif test edilebilmesi için Swagger OpenAPI dokümantasyonunun entegre edilmesi.

Yapılamayanlar ve Gerekçeleri

1. Egzersiz ve Yakılan Kalori Takip Modülü: Kullanıcının yaptığı spor aktivitelerini girerek yaktığı kaloriyi günlük bütçesinden düşmesini sağlayan ek egzersiz modülü. Proje isterlerinde öncelikli olarak besin tüketimi (kalori alımı) ve makro takibine odaklanılması talep edildiği için, egzersiz takip fonksiyonu projenin ana kapsamı dışında tutulmuş ve gelecek çalışmalar (future scope) olarak planlanmıştır.
2. Gelişmiş Grafik ve İstatistik Analiz Paneli: Kullanıcının haftalık veya aylık kalori değişimlerini pasta veya çizgi grafiklerle analiz edebileceği görsel raporlama paneli. JavaScript tabanlı grafik kütüphanelerinin (örn. Chart.js) entegrasyonu ve veritabanı analiz sorgularının optimizasyonu ek geliştirme süresi gerektirdiğinden, proje teslim takvimi doğrultusunda bu özellik ertelenmiştir.
3. Kullanıcı Şifre Sıfırlama ve E-posta Doğrulama Modülü: Kullanıcıların şifrelerini unutmaları durumunda e-posta aracılığıyla şifre sıfırlama bağlantısı alabilmesini sağlayan sistem. Uygulamanın yerel sunucuda (localhost) çalışması ve harici bir SMTP e-posta sunucusu kurulumunun projenin temel veri yönetim kapsamını aşması nedeniyle bu güvenlik özelliği devre dışı bırakılmıştır.

6. Proje Sonucu

Bu proje, okulda teorik olarak gördüğümüz birçok yazılım kavramını pratiğe dökme şansı bulduğum ve gerçek bir ürün geliştirme sürecini baştan sona deneyimlediğim çok değerli bir çalışma oldu. Proje sayesinde ASP.NET Core framework'ünü kullanarak sıfırdan çalışan bir web uygulaması geliştirme sürecini deneyimledim. Özellikle MVC (Model-View-Controller) mimarisini uygulayarak; arayüz (View), veri modelleri (Model) ve iş mantığı (Controller) arasındaki sorumlulukların nasıl ayrılması gerektiğini ve bu ayrımın büyük projelerde kodun okunabilirliğini ne kadar artırdığını bizzat uygulayarak kavradım. Bunun yanı sıra, Dependency Injection (Bağımlılık Enjeksiyonu) kavramının sadece teoride kalmayıp, veritabanı bağlantı nesnelerimizi (CalorieTrackerContext) yönetirken sınıflar arası bağımlılığı azaltmada ne kadar kritik bir rol oynadığını gördüm. Entity Framework Core ve SQLite kullanarak veritabanı şemalarının otomatik yönetilmesi ve veri erişim süreçlerinin kolaylaştırılması konularında önemli pratik beceriler kazandım.

Bu dersin bana en büyük katkısı, yazılım geliştirmede sadece kod yazmayı değil, bir projeyi sürdürülebilir ve endüstri standartlarında tasarlamayı öğretmesi oldu. RESTful API tasarımı ve Swagger OpenAPI entegrasyonu sayesinde, yazdığımız backend servislerinin dış dünyaya nasıl açıldığını ve standartlara uygun olarak nasıl belgelendiğini öğrendim. Postman kullanarak yaptığım API testleri ise arka plan servislerinin güvenilirliğini sınama konusunda bana sistemli bir test bakış açısı kazandırdı. Bir web uygulamasının güvenlik (kimlik doğrulama), veritabanı yönetimi ve kullanıcı arayüzü gibi farklı katmanlarının bir bütün halinde nasıl organize edildiğini kavramak, gelecekteki yazılım kariyerim için bana çok sağlam bir temel oluşturdu.

7. Kaynakça

1. [Microsoft / Microsoft Learn, "Overview of ASP.NET Core MVC", https://learn.microsoft.com/en-us/aspnet/core/mvc/overview, Erişim Tarihi: 7 Haziran 2026.
2. [Microsoft / Microsoft Learn, "SQLite Database Provider - EF Core", https://learn.microsoft.com/en-us/ef/core/providers/sqlite/, Erişim Tarihi: 7 Haziran 2026.
3. [Microsoft / Microsoft Learn, "Use cookie authentication without ASP.NET Core Identity", https://learn.microsoft.com/en-us/aspnet/core/security/authentication/cookie, Erişim Tarihi: 7 Haziran 2026.
4. [Swashbuckle / GitHub Repository, "Swashbuckle.AspNetCore - Swagger tools for documenting APIs built on ASP.NET Core", https://github.com/domaindrivendev/Swashbuckle.AspNetCore, Erişim Tarihi: 7 Haziran 2026.
5. [BCrypt.Net / GitHub Repository, "BCrypt.Net-Next - Secure password hashing library for .NET", https://github.com/BcryptNet/bcrypt.net, Erişim Tarihi: 7 Haziran 2026.

— Raporun Sonu —

İstanbul Beykent Üniversitesi | 2025–2026 Bahar Dönemi